

Modern Reinforcement Learning: Project Report

Value smoothing using similarity-based on latent embeddings

Aniket A W

Ishan C

Thiruvankadam S R

Introduction

The effectiveness of reinforcement Learning in high dimensional state spaces depends on the quality of representations learned by the network during training. It is often difficult to improve the convergence properties of such training methods. Though the nature of the Bellman operator provides theoretical guarantees of convergence, there is no easy way to exploit the latent structure of the environment to speed up the learning process. The Bellman update may be slower to converge to a good policy without structured exploration in sparse reward settings.

There are different areas of research in reinforcement learning trying to speed up learning. Parallel learning methods by making asynchronous updates to policy networks have been shown to speed up convergence. Mnih et al. [1] used asynchronous updates to actor-critic methods to half the training time using a multi-core CPU for Atari games. Clemente et al. [2] proposes a general framework for parallel reinforcement learning on a single machine where actions generate different copies of the environment from the same policy providing for more stable learning.

Another side of looking at the speed of the learning problem is sample efficiency. While it is possible to generate many samples through a simulator, it might be difficult in real-life applications to generate a large number of samples until the agent reaches reasonable performance levels. It is also nontrivial to create a perfect simulation representing accurate transition dynamics that can enable sufficient sample collection to ensure the learning of a good policy. Since most reinforcement learning involves the agent acting based on visual information, learning better state representations from images in a sample efficient manner has also been shown to improve the sample efficiency of the agent's training. Yarats et al. [3] jointly learned a state representation using autoencoders with an auxiliary loss in a sample efficient manner that helped achieve good performance even under noisy observation settings. [Mico] used bisimulation metrics to learn a state similarity measure for better latent representations

and showed it to improve the performance of value-based methods and policy gradient methods like SAC.

The vanilla DQN used an epsilon noise in the behavior policy to explore outside of the policy. However, this does not guarantee to find highly valued or novel states that can help speed up convergence. More sophisticated methods such as NovelD [4] and Go-Explore [5] have been proposed for structured exploration of novel or potentially high valued states by using intrinsic rewards that can help train a good policy, especially under sparse reward settings. Alternatively, sample efficiency can also be enhanced through good exploration techniques that equip the agent to find high-value states sooner, resulting in faster policy convergence.

Building on top of this, in this work, we aim to speed up the learning process by updating the Q values of states using information from the Q values of other sampled states along with a learned distance metric. We also hope to use this learned distance metric to enable better-structured environment exploration, leading to faster convergence and learning an overall better policy.

Related work

In MICO [6], the authors learn a bisimulation metric as an auxiliary task to estimate state similarity. They discuss the limitations of learning a bisimulation metric using Wasserstein distance in terms of the computational complexity and propose an alternate formulation based on independent couplings of distributions. The bisimulation metric is the operator defined as follows:

$$T_k(d)(x, y) = \max_{a \in A} [|r_x^a - r_y^a| + \gamma W_d(P_x^a, P_y^a)] \quad (1)$$

The similarity in the embedded space corresponds to the similarity in the rewards and the similarity in the states they transition to. The MICO distance is defined as a diffuse metric since it allows $d(x, y) > 0$ even for $x = y$ as opposed to a regular metric. And the MICO loss is defined as making the distance between state representations learned by the network as close to the MICO distance as possible.

$$L_{MICO}(w) = E_{\langle x, r_x, x' \rangle, \langle y, r_y, y' \rangle} [(T_w^U(r_x, x', r_y, y') - U_w(x, y))^2] \quad (2)$$

NovelD uses regulated Novelty Difference (NovelD) of consecutive states in a trajectory, allowing the agent to explore unexplored regions in a breadth-first search, weighing unknown regions equally. Intuitively, the algorithm provides a large intrinsic award at the explored and new region boundary.

The intrinsic reward that guides exploration for NovelD is based on the equation

$$r(s_t, a_t, s_{t+1}) = \max [\text{novelty}(s_{t+1}) - \alpha * \text{novelty}(s_t), 0] \quad (3)$$

Where novelty is determined similar to RND using the difference between a random fixed target network and a trainable predictor network as

$$\text{novelty}(s_t) = \text{novelty}(s_t; w) = ||\phi(s_t) - \phi'_w(s_t)||_2 \quad (4)$$

In addition, RIDE [14] avoids back-and-forth state visiting repetition by scaling the intrinsic reward by the inverse of the episodic state visitation counts. This approach to novelty exploration leads to greater rewards for trajectories moving out of explored regions and better asymptotic consistency.

Go-explore works on the principle of remembering states that have been visited, returning to explore from promising states, and solving simulated environments by exploiting any available means, allowing for better agent performance against problems such as *derailment* and *detachment*.

Detachment refers to the idea of being detached from the frontiers of high intrinsic reward (IR) by an agent-driven by intrinsic motivation (IM). The principle is that after consuming the IR of an area, the agent may move to another region and later not be able to return to the original site, which might still have a high potential IR remaining. This is because the IR of the path leading to that frontier may have already been consumed. To address this, Go-Explore remembers and returns to previously discovered promising areas for exploration.

Derailment can occur when an agent's stochastic perturbations do not allow it to return to states with high IR. This is often the result of basic exploration mechanisms that rarely replicate the precise actions needed to reach that state. An example of such a technique would be epsilon-greedy exploration, which is often used in conjunction with IM. Go-Explore deals with this by first returning to the promising state before continuing to explore further.

The Go-Explore algorithm works in two phases. First, it explores the state space and builds an archive of interestingly different game states. Second, it chooses a cell (game state) from the archive to explore from. This choice could be made uniformly at random. Although using a heuristic approach of preferring some cells has improved performance.

The mentioned approaches so far have shown state-of-the-art performance in exploration. However, they lack two primary aspects. One, they don't leverage the similarity between states for quicker and more robust convergence. Second, using more straightforward machine learning techniques in place of neural networks in the initial phases can lead to faster convergence as the latter require greater amounts of data to train. In GoExplore and NovelD, novelty is still state visitation and intrinsic reward-based instead of taking advantage of the state space distribution. Though the MICO distance helps shape state similarity, this is never explicitly taken advantage of to use as additional information in learning Q-values of other states. Instead, we aim to explicitly use this known distance for better Q-value estimation, especially when visiting newer forms, and for a more structured exploration by using latent state representations.

Background

Q Learning:

Q-Learning is a value iteration algorithm that uses iterative updates to the Bellman equation until the values converge, as given by the following equation:

$$Q_{i+1}(s, a) = \mathbb{E} [(r + \gamma * \max_{a'} Q_i(s', a')) | s, a]$$

Eventually, this value iteration converges to the optimal Q values $Q^*(s, a)$ as $I \rightarrow \infty$. However, more practical algorithms use a function approximation to represent the Q value function, typically a neural network [DQN] $Q(s, a)$. This helps with faster convergence and makes this type of learning possible in environments with high dimensional states.

A neural network of this form can be trained by using a squared error loss function between the estimated Q-values and the actual reward.

$$Loss(Q(s, a; \theta)) = Q(s, a; \theta) - [r + \gamma * Q(s', a'; \theta)]$$

This loss is then optimized by using stochastic gradient descent. To account for the deadly triad in the presence of function approximation, TD learning, and off-policy learning, approaches like DQN use a replay buffer to sample states that break the correlation between states in a trajectory and enable learning in these settings.

Prioritized replay and exploration

[P replay paper] introduced using a prioritized replay buffer that provides higher sampling probabilities to states with higher Q-error. The intuition is that states with higher Q-error have more information to offer and will also speed up training. An

importance sampling weight is assigned to states sampled from the buffer, and this weight is used when calculating the loss.

$$w_i = \left(\frac{1}{N} \frac{1}{P(i)}\right)^\beta$$

Where β is the importance weighting factor and the sampling probability is directly proportional to the Q learning error

$$P(i) \propto (Q(s, a) - [r + \gamma Q(s', a')])$$

Similarity and Distance measures

Understanding similarity between states is a non-trivial problem. Comparing two states' visual representations is insufficient as states that are visually different be almost identical. For example, in atari space invaders, as each spaceship moves slightly, each frame is vastly different. However, we understand them to be very similar in nature. Our proposed solution is to use embeddings of neural networks that understand the nature of the games. These embeddings contain relevant information about states, which we then use to compare similarities.

We propose embeddings of either the bottleneck layer of an auto-encoder or the feature vector from the coating after the convolutional layers of the DQN.

To compare similarity between the embedding, we propose using one of three similarity measures.

1. Cosine Similarity

$$D(A, B) = \frac{A \cdot B}{|A||B|}$$

2. L1 Norm

$$D(A, B) = \sum_{i=1}^n |A_i - B_i|$$

3. L2 Norm

$$D(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

Methods

Value smoothing based on state similarity

In this work, we propose to use a similar distance metric to normalize the Q-value estimate with Q-values of randomly sampled states with a weight inversely proportional to their distance from the current state. We believe this will help in more stable estimates of Q-values and lead to faster convergence. We propose to change the Q-value update as follows.

$$Q(s, a) = Q(s, a) + \alpha (r + \gamma \operatorname{argmax}_{a'} Q(s', a')) + \frac{\beta}{|S|} \sum_{s'' \in S} \frac{\operatorname{argmax}_{a''} Q(s'', a'')}{d(s, s'')} \quad (5)$$

Where S is a randomly sampled set of states from the experience replay and β is a decay term that will reduce the weight of the stabilization term over the length of training. One possible distance measure is the cosine similarity equation [\(6\)](#).

$$d(s_1, s_2) = \frac{s_1^e \cdot s_2^e}{\|s_1^e\| \|s_2^e\|} \quad (6)$$

Representation learning/Latent Embeddings extraction

The first step of the process is to enable states' a suitable representation. While using the raw pixel input is an option, this is high dimensional and might be very noisy. Instead, we use representation learning to embed the state into a feature vector used to compute the distance between states.

There were two possible ways to do this. One is to train an Auto-Encoder [ref] to reconstruct states and extract informative features learned by its bottleneck layer. Auto-Encoders work on the principle of compression of input through a bottleneck layer, generating lower dimensional embeddings that are then reconstructed to create the input again as the output. This allows the network to extract information from the input in a lower-dimensional space while reducing information that is not useful and highly correlated. Auto-encoders might typically require a good amount of data to train, which can be acquired by letting a random-action policy agent roam around in an environment. While this can give a decent set of features, the states explored by a random agent do not represent the whole environment, which might be an issue when using this for the entire training.

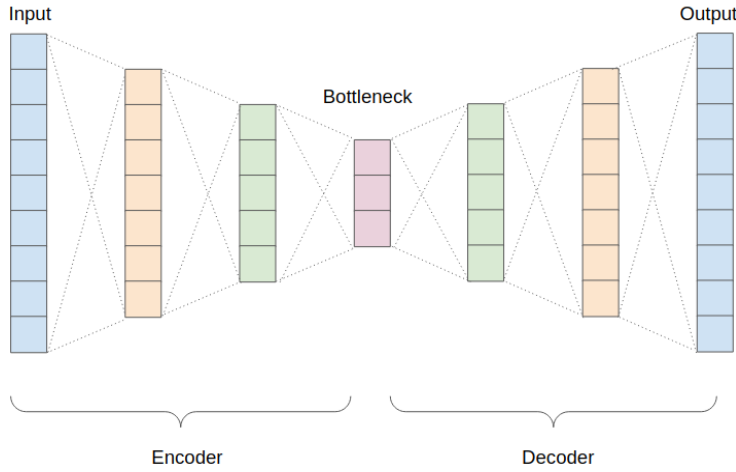


Figure 1: A conceptual depiction of the bottleneck architecture

The other option is to use the features extracted from the last embedding layer of the Deep Q-Learning Network [ref]. The DQN comprises a neural network that learns the Q-values of an agent through the Q-Learning framework. The network first extracts useful information through convolutional layers that remove increasingly complex features with the depth of each layer. This is followed by a fully-connected layer that predicts Q-values based on the feature information extracted by its preceding layers. An advantage of this approach is that the network extracts features directly linked to the output Q-Values of the agent in the same environment. Thus, its embeddings would contain features that would provide an informative metric to similar states. We note that extracting such features would take many steps, and the network of This might not have very informative features at the beginning of the training. Still, the network eventually learns a good feature representation of the states that aids the smoothing update.

Prioritized replay based on similarity for exploration

To use the state similarity for exploration, we use a prioritized replay buffer where the priorities are proportional to the distance from the sampled states.

$$P(i) \propto \sum_{s''} d(s, s'')$$

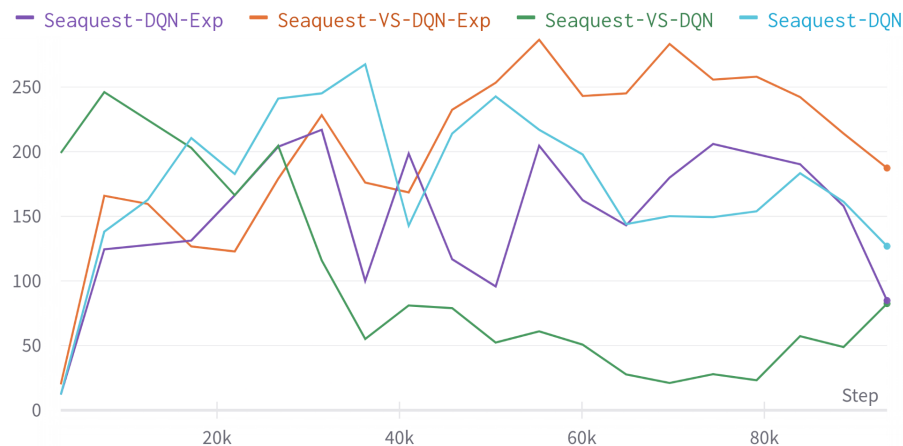
We do not use the Q learning errors for the priorities since that has already been shown to perform better than DQN, and since we are comparing our results against the vanilla DQN, this might make the comparison unfair.

Experiments

Atari Environment

We compared the performance of DQN with value smoothing (VS-DQN), DQN with the exploration based on similarity (DQN-Exp), a combination of the two VS-DQN-Exp to the performance of vanilla DQN as the baseline. We tested the performance of these methods in 3 Atari environments Space Invaders, Breakout, and Seaquest. Being environments with high dimensional states with a rich state space provided a good testbed for the experiments.

The experiment setup was to train each of these algorithms for 100k steps in each environment and measure their performance. An evaluation was done for every 5000 steps of training by running 20 episodes and averaging their scores. The results of these experiments are shown in the figure.



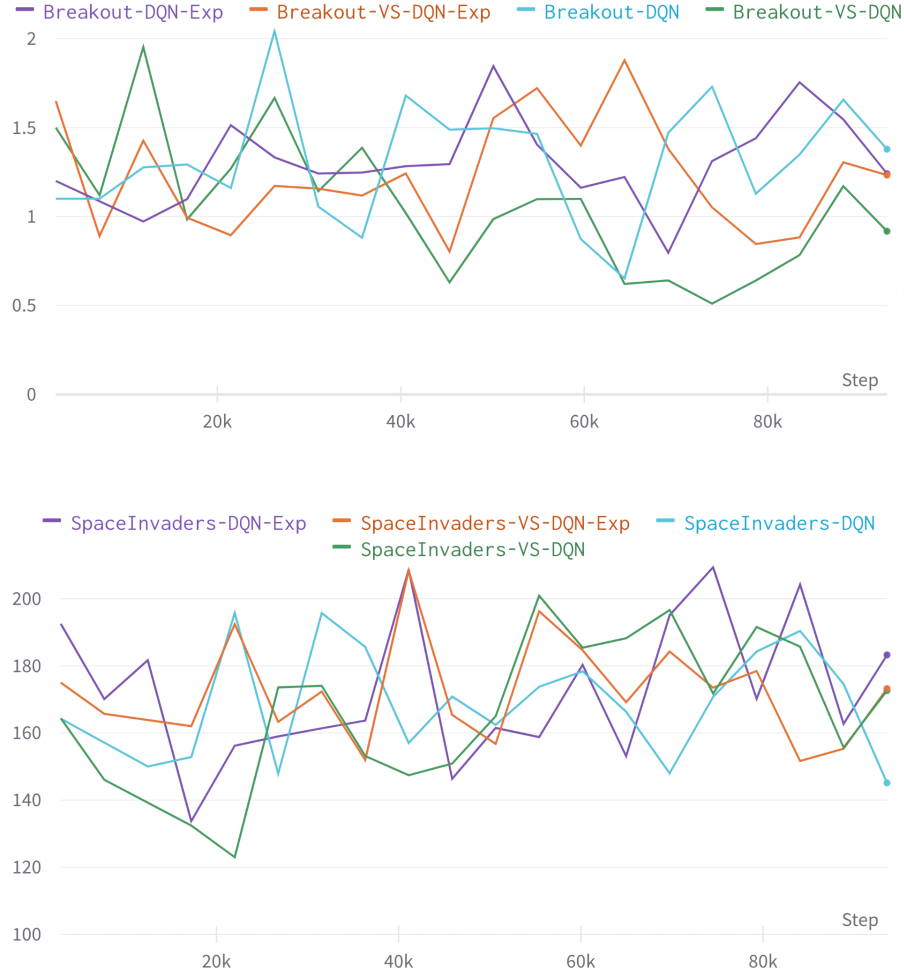


Figure 2. Experimental results for Seaquest, Space Invaders and Breakout environments comparing DQN, VS-DQN, VS-DQN-Exp and DQN-Exp.

Discussion

In our experiments on Seaquest, we found that Sequest with VS and exploration had the best performance, even outperforming standard DQN over the entire course of the experiment. Sequest with VS outperformed vanilla DQN in the earlier course of training. However, it failed after the initial stages. We hypothesize that although the similarity metric allows for quicker convergence, once it has trained sufficiently, it might interfere with achieving its maximal performance as per the Bellman Equation. In SpaceInvaders, the performance across all algorithms was similar, and we didn't find anything conclusive from the results. In Breakout, we again found greater performance by VS-DQN over Vanilla DQN in the initial stages, with the former's rewards fizzling out towards the end. We note that our results are only a preliminary investigation into a

novel approach and do not form a conclusive comparison between the noted algorithms. These results are indicative but do not provide sufficient information due to variance. In future experiments, we could average out performance to lower variance and provide more conclusive results.

Additional Analysis

Impact of smoothing on training time:

As we are performing additional computation per optimization step during training, it is expected that there will be an added overhead. We compared the time taken per step of training for the vanilla DQN and VS-DQN-Exp algorithms over the first 10000 training steps, and the results are in the figure below.

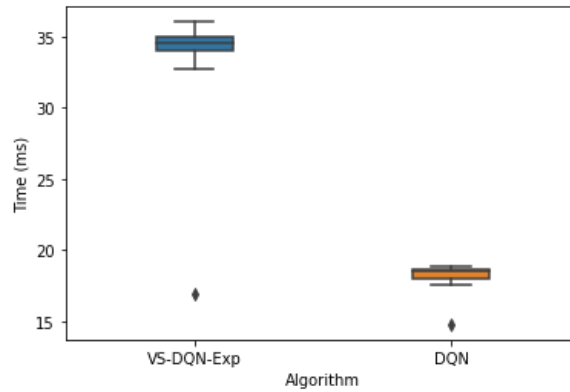


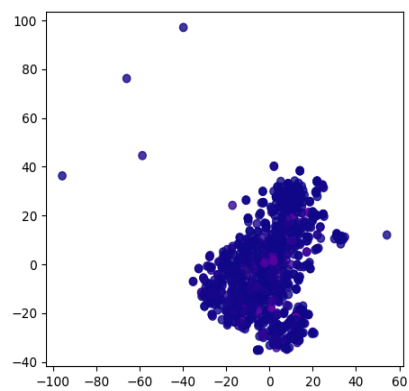
Figure 3. Optimization step time for the vanilla DQN update vs the smoothed update.

We observed that the smoothed update takes about 1.8 times more time than the vanilla DQN update on average.

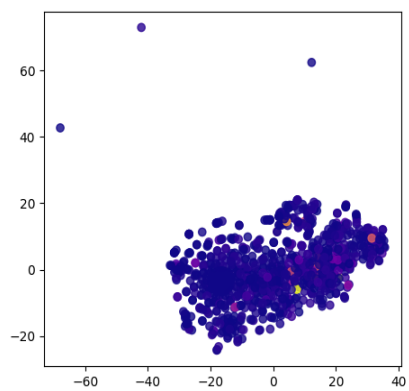
Faster value propagation:

While initial q-values of the DQN are slow to converge, value smoothening allows the q-values to quickly converge, as the updated q-value from a state with a high reward propagates to other similar states. To demonstrate this, we trained both of these algorithms for 10000 steps, sampled 1000 states for the time steps - 1500, 3500, 7500, and 9000 and analyzed their value propagation. We extracted their DQN feature embeddings and visualized their state values using t-stochastic neighborhood embeddings.

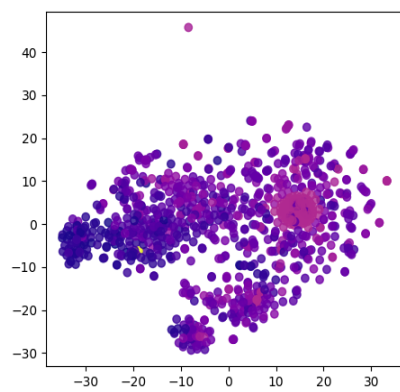
As we can see from the figure, the smoothed state space has a much better value propagation than the vanilla DQN.



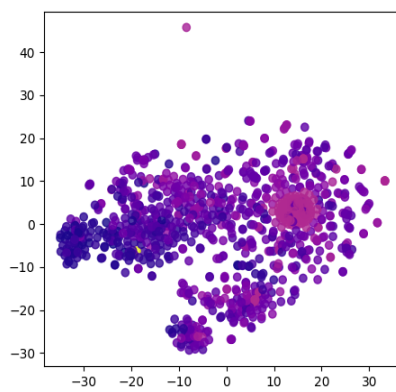
(i)



(ii)

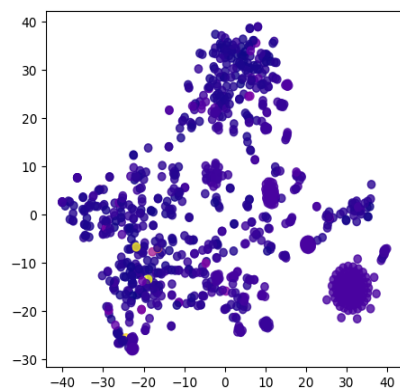


(iii)

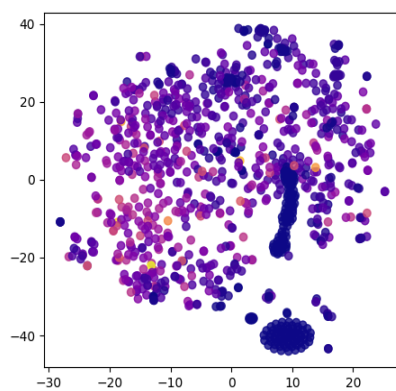


(iv)

(a) DQN



(i)



(ii)

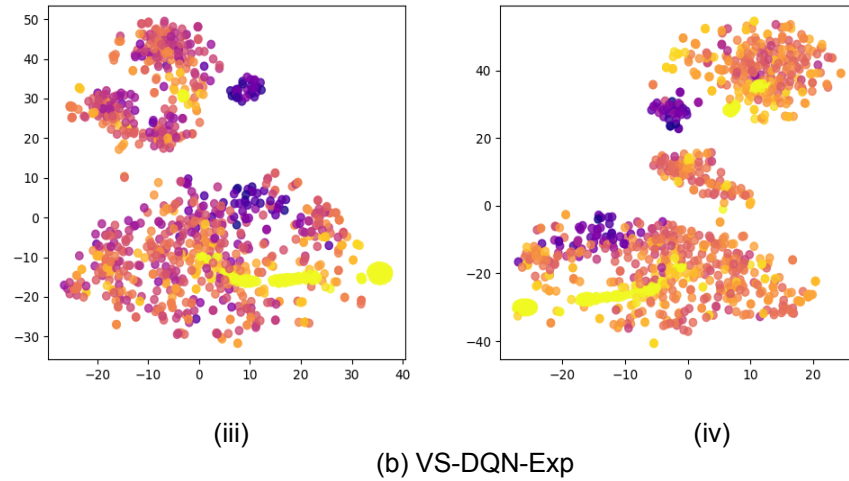


Figure 4. Propagation of Q-values in sampled states during the first 10000 steps of training

Conclusion

We found that in the initial stages of test rewards, we achieved good test results, with our method showing faster convergence in two environments: Seaquest and Breakout. In the third environment, Space-Invaders, our model performed on par with vanilla DQN. However, more tests are required to ascertain the efficiency of our method. Additional distance metrics can be explored, which could lead to better convergence. New architectural changes could be tested to find better representations for structural similarity. Analyzing different decaying or annealing methods to tune our similarity metric's influence on the overall Q-Value can also help resolve the issue of our method's accuracy falling over large training steps.

- Method shows potential
- Could perform better, with better hyperparameter tuning, etc.
- Exploration can be extended to state resetting scenarios